

LAB 3: MONITORING, LOGGING & DEVSECOPS

Name- Manya Singh

Roll no – 2301010389

Course – B.TECH Cse-6

Question 1 – Monitoring using Prometheus and Grafana

Objective: Set up monitoring for a simple Express.js application using Prometheus for metrics collection and Grafana for visualization. The application is containerized using Docker and exposes a /metrics endpoint for Prometheus scraping.

1. Project Setup

Create a separate project.

```
mkdir lab3_monitoring  
cd lab3_monitoring  
mkdir app  
cd app  
npm init -y  
npm install express
```

The initial application contains a home route, a health-check route, and a simple users API.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\MANYA SINGH> mkdir lab3_monitoring

Directory: C:\Users\MANYA SINGH

Mode                LastWriteTime         Length Name
----                -
d-----          05-10-2026    15:23                lab3_monitoring

PS C:\Users\MANYA SINGH> cd lab3_monitoring
PS C:\Users\MANYA SINGH\lab3_monitoring> mkdir app

Directory: C:\Users\MANYA SINGH\lab3_monitoring

Mode                LastWriteTime         Length Name
----                -
d-----          05-10-2026    15:24                app

PS C:\Users\MANYA SINGH\lab3_monitoring> cd app
PS C:\Users\MANYA SINGH\lab3_monitoring\app> npm init -y
Wrote to C:\Users\MANYA SINGH\lab3_monitoring\app\package.json:

{
  "name": "app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\MANYA SINGH\lab3_monitoring\app> npm i express

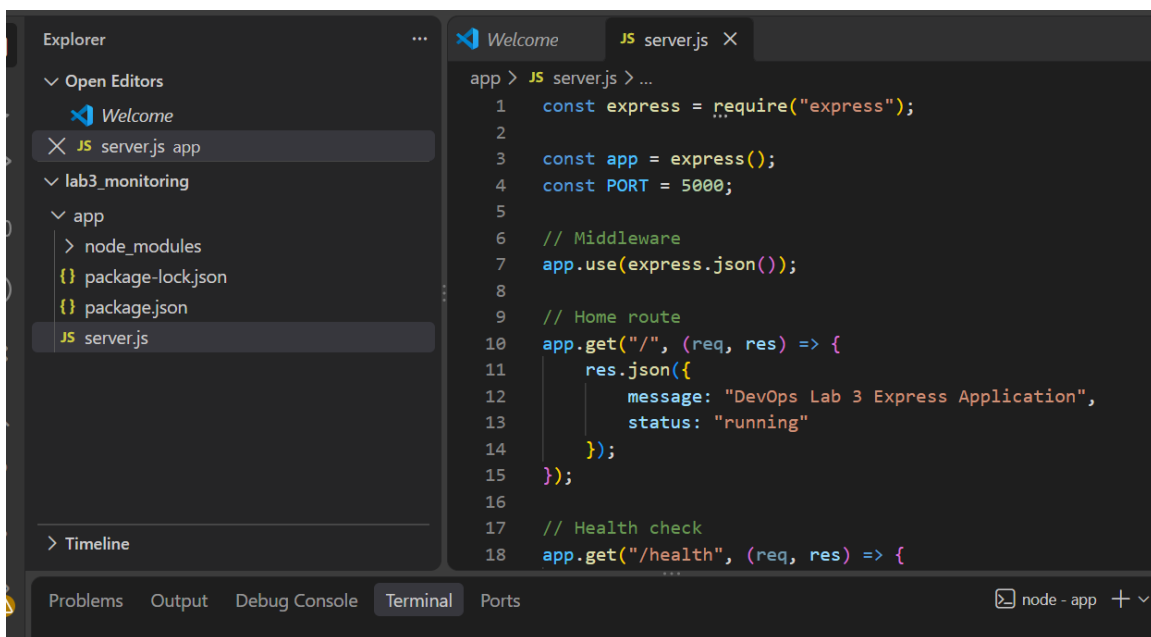
added 68 packages, and audited 69 packages in 6s

28 packages are looking for funding
  run `npm fund` for details

```

2. Create the Express Application

Create app/server.js with the following application structure:



```

app > JS server.js > ...
1  const express = require("express");
2
3  const app = express();
4  const PORT = 5000;
5
6  // Middleware
7  app.use(express.json());
8
9  // Home route
10 app.get("/", (req, res) => {
11   res.json({
12     message: "DevOps Lab 3 Express Application",
13     status: "running"
14   });
15 });
16
17 // Health check
18 app.get("/health", (req, res) => {

```

Run the application with `node server.js` and verify `http://localhost:5000`, `http://localhost:5000/health`, and `http://localhost:5000/api/users`.

3. Dockerize the Express Application

Create `app/Dockerfile`

Create `app/.dockerignore`:

```
node_modules
npm-debug.log
```

Build and run the Docker image:

```
docker build -t lab3_monitoring-app .

docker run -d --name lab3-monitoring-app -p 5000:5000 lab3_monitoring-app

docker ps

docker logs lab3-monitoring-app
```

```
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app> docker build -t lab3_monitoring-app .
=> => exporting manifest list sha256:27c210dc3d1d70334051e0448bdd36799269f623cf375f13d1f892344c131c3e
=> => naming to docker.io/library/lab3_monitoring-app:latest
=> => unpacking to docker.io/library/lab3_monitoring-app:latest
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app>

PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app> docker run -d --name lab3-monitoring-app -p 5000:5000 lab3_monitoring-app

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
0d7a0871785f	lab3_monitoring-app	"docker-entrypoint.s..."	About a minute ago	Up About a minute	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

```
lab3-monitoring-app
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app>
```

4. Add Prometheus Instrumentation

Install the Prometheus client library:

```
npm install prom-client
```

Update `server.js` to create a Prometheus registry, collect default Node.js metrics, count HTTP requests, and expose the `/metrics` endpoint.

Rebuild the image and recreate the container after modifying `server.js`. Verify the endpoint at <http://localhost:5000/metrics>.

```
localhost:5000/metrics

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 0.195145

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 0.094796

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 0.289941

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1791195569

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 54886400

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 684154880

# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes 53014528

# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 21

# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1048576
```

5. Configure Prometheus

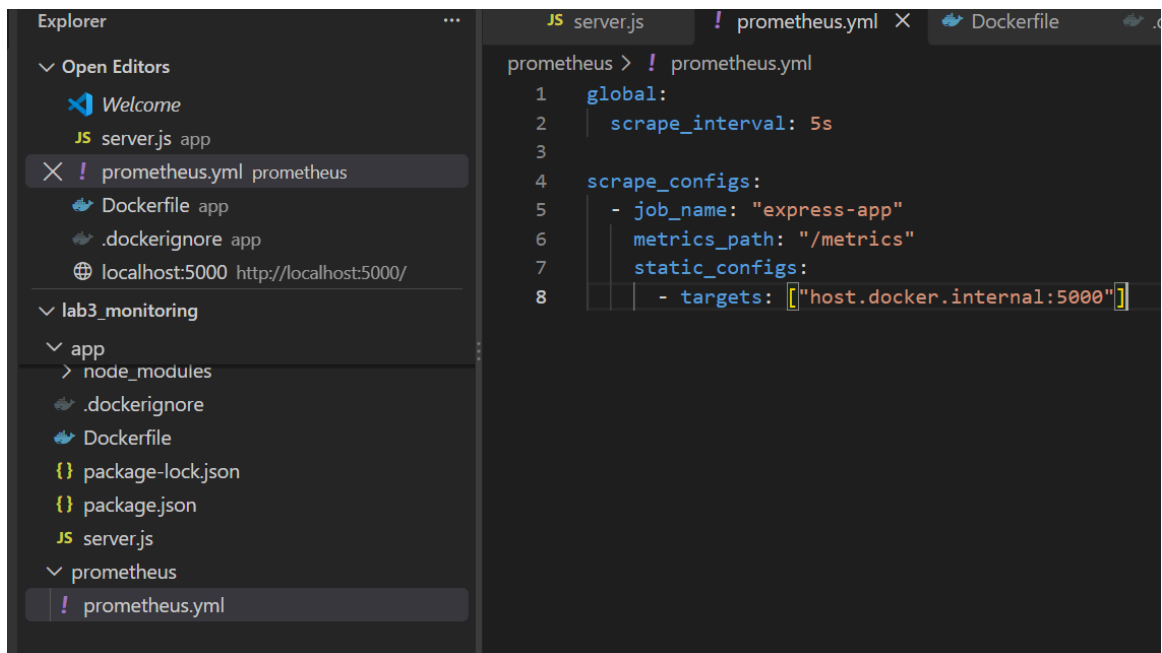
From the lab3_monitoring root directory, create a prometheus folder and add prometheus/prometheus.yml:

```
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring\app> cd..
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring> mkdir prometheus

Directory: C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring

Mode                LastWriteTime         Length Name
----                -
d-----          05-10-2026    15:52                prometheus

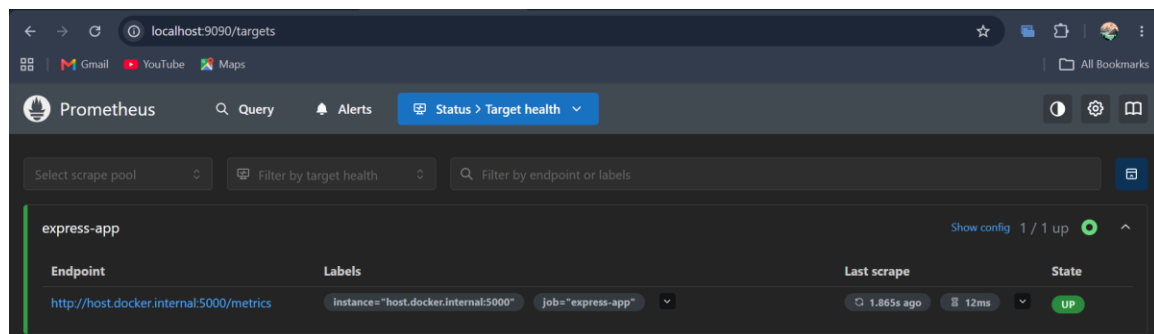
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring>
```



Run Prometheus in Docker:

```
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring> docker run -d `
>> --name prometheus `
>> -p 9090:9090 `
>> -v "${PWD}\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml" `
>> prom/prometheus
```

Open <http://localhost:9090> and navigate to Status → Target health. The express-app target should show UP.

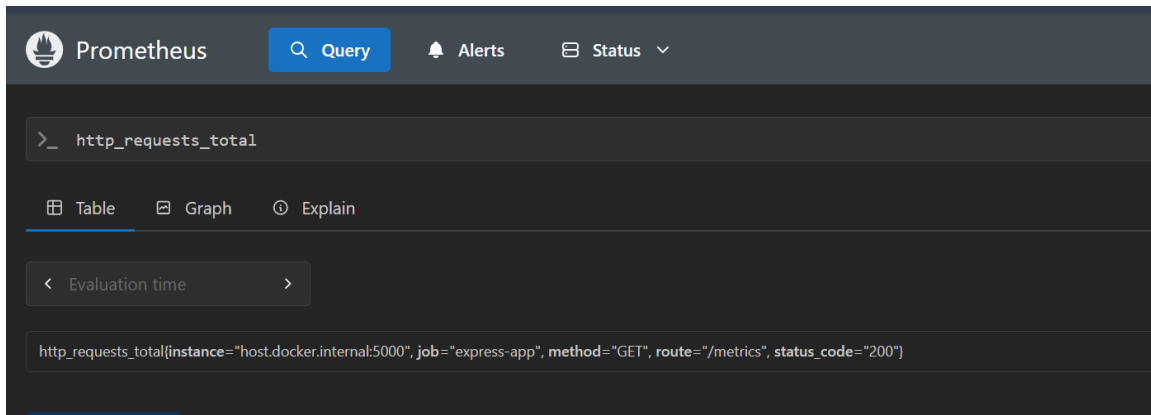


6. Verify Prometheus Metrics

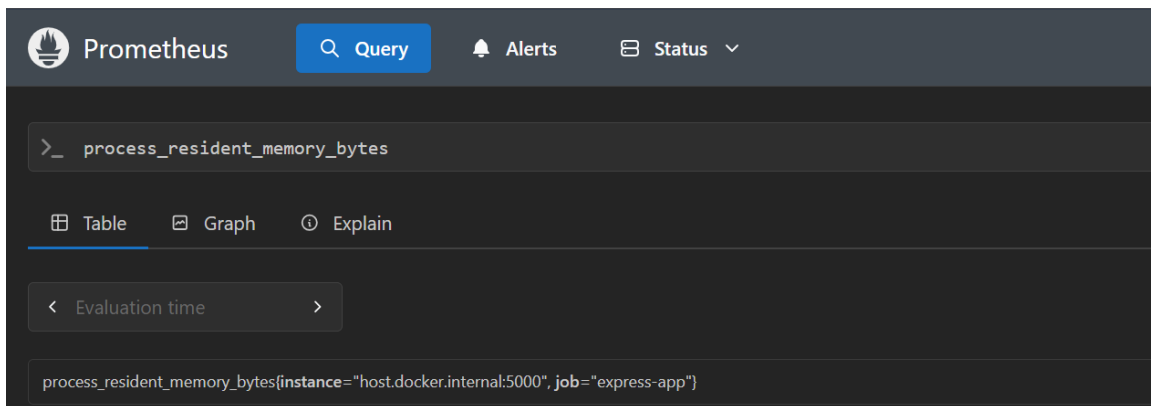
Use the Prometheus Query page to verify the following queries:

- `http_requests_total`
- `process_resident_memory_bytes`
- `rate(http_requests_total[1m])`

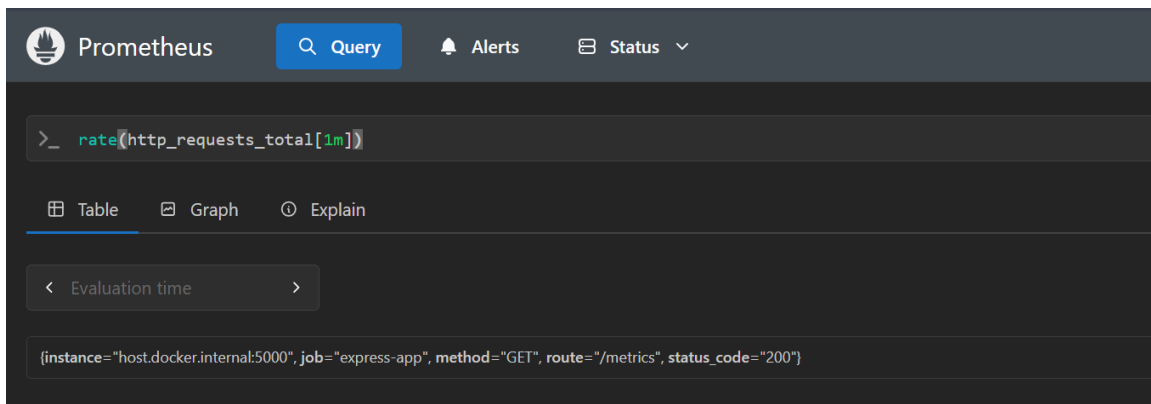
Generate traffic by opening the Express routes several times, then execute the queries again to observe updated metrics.



The Prometheus Query Editor interface shows a query for `http_requests_total`. The query is entered in the text box, and the 'Table' view is selected. Below the query, the evaluation time is shown as 'Evaluation time'. The query result is displayed as a table with one row: `http_requests_total{instance="host.docker.internal:5000", job="express-app", method="GET", route="/metrics", status_code="200"}`.



The Prometheus Query Editor interface shows a query for `process_resident_memory_bytes`. The query is entered in the text box, and the 'Table' view is selected. Below the query, the evaluation time is shown as 'Evaluation time'. The query result is displayed as a table with one row: `process_resident_memory_bytes{instance="host.docker.internal:5000", job="express-app"}`.



The Prometheus Query Editor interface shows a query for `rate(http_requests_total[1m])`. The query is entered in the text box, and the 'Table' view is selected. Below the query, the evaluation time is shown as 'Evaluation time'. The query result is displayed as a table with one row: `{instance="host.docker.internal:5000", job="express-app", method="GET", route="/metrics", status_code="200"}`.

7. Set Up Grafana

Run Grafana in Docker:

```
docker run -d --name grafana -p 3001:3000 grafana/grafana
```

Open <http://localhost:3001> and sign in with the initial Grafana credentials `admin/admin`, changing the password if prompted.

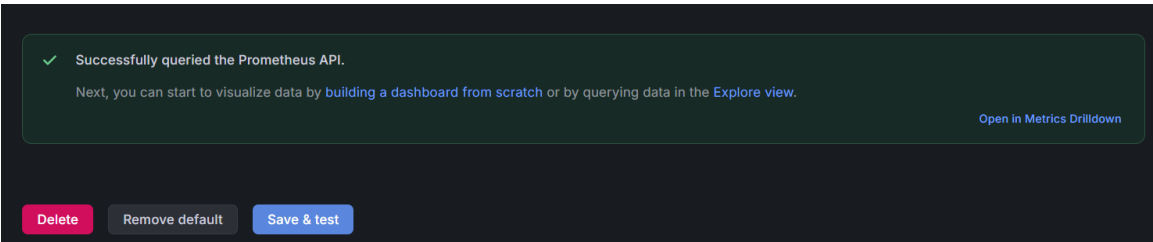
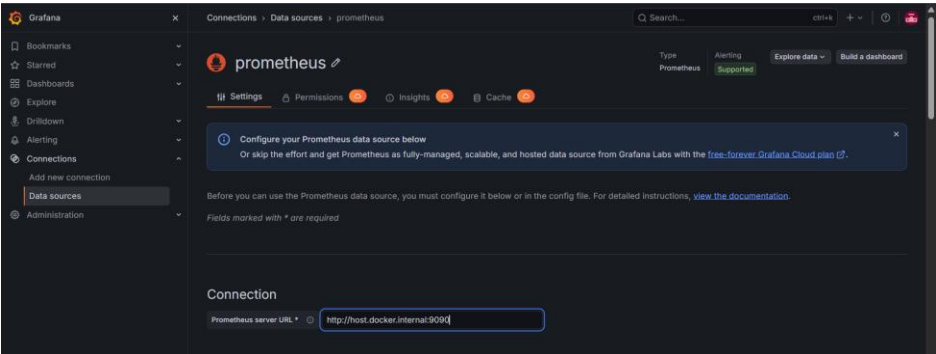
```
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring> docker run -d --name grafana -p 3001:3000 grafana/grafana
bb9dbccd4085: Pull complete
0ea8972a4a11: Pull complete
4f4fb70ef54: Pull complete
Digest: sha256:b28bae15e219c998fb0e0424ed724930cc61b1f61fb404d47c862f9a23f9e572
Status: Downloaded newer image for grafana/grafana:latest
583f4a88cf879ada8d1b0bc695339ac32018ca1a1f46b3662759c30b206d2a3c
PS C:\Users\MANYA SINGH\OneDrive\Documents\lab3_monitoring>
```

8. Connect Grafana to Prometheus

In Grafana, open Connections → Data sources → Add new data source → Prometheus. For the URL, use:

```
http://host.docker.internal:9090
```

Click Save & test and verify that Grafana successfully queries the Prometheus API.



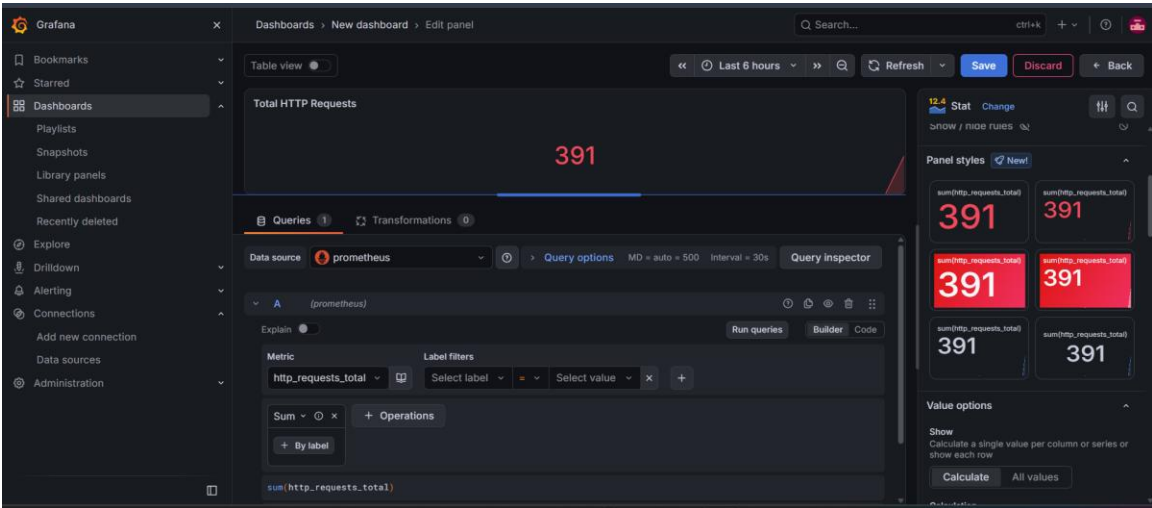
9. Create the Grafana Monitoring Dashboard

Create a dashboard named: Lab 3 - Application Monitoring.

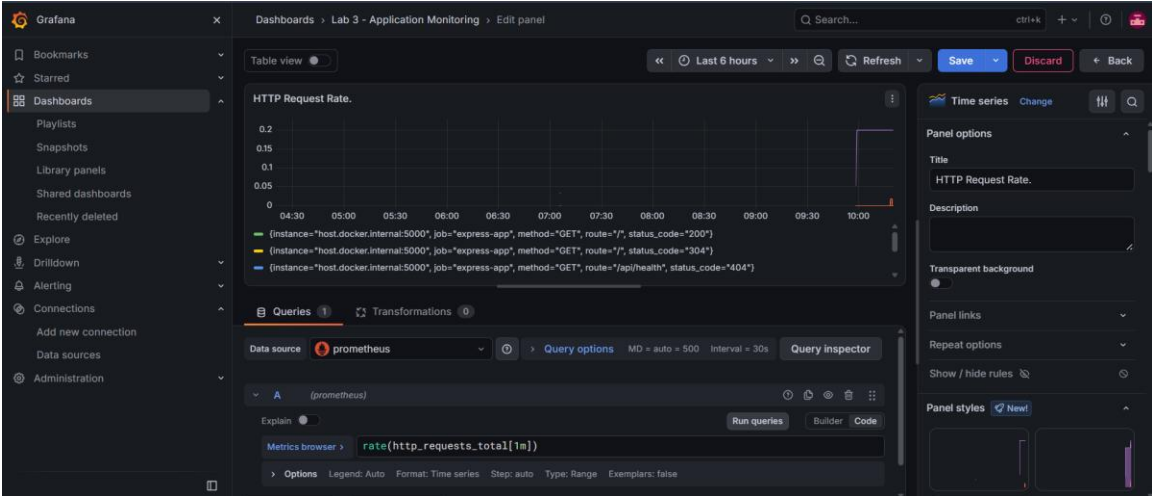
Five panels were created with the following queries and visualization types:

Panel	PromQL Query	Visualization	Purpose
Total HTTP Requests	sum(http_requests_total)	Stat	Shows total request count
HTTP Request Rate	rate(http_requests_total[1m])	Time series	Shows request rate over time
Backend Memory Usage	process_resident_memory_bytes	Time series	Shows backend memory usage
HTTP Requests	sum by (status_code)	Time series	Shows requests

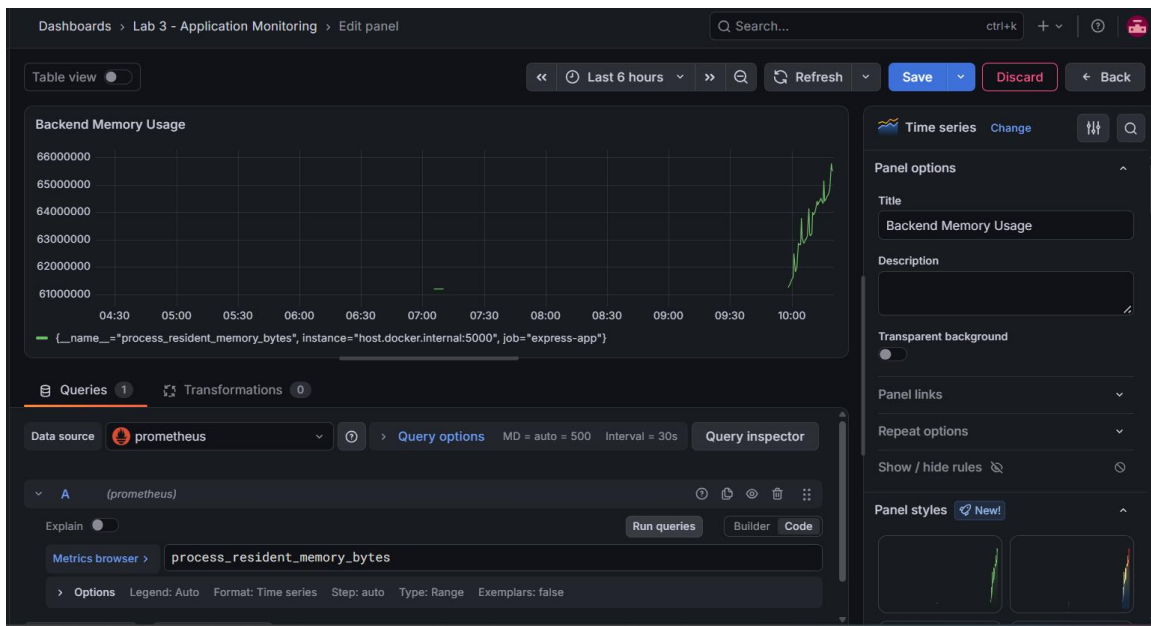
by Status Code	(http_requests_total)		grouped by status
Requests by Endpoint	sum by (route) (http_requests_total)	Bar chart	Shows requests grouped by endpoint



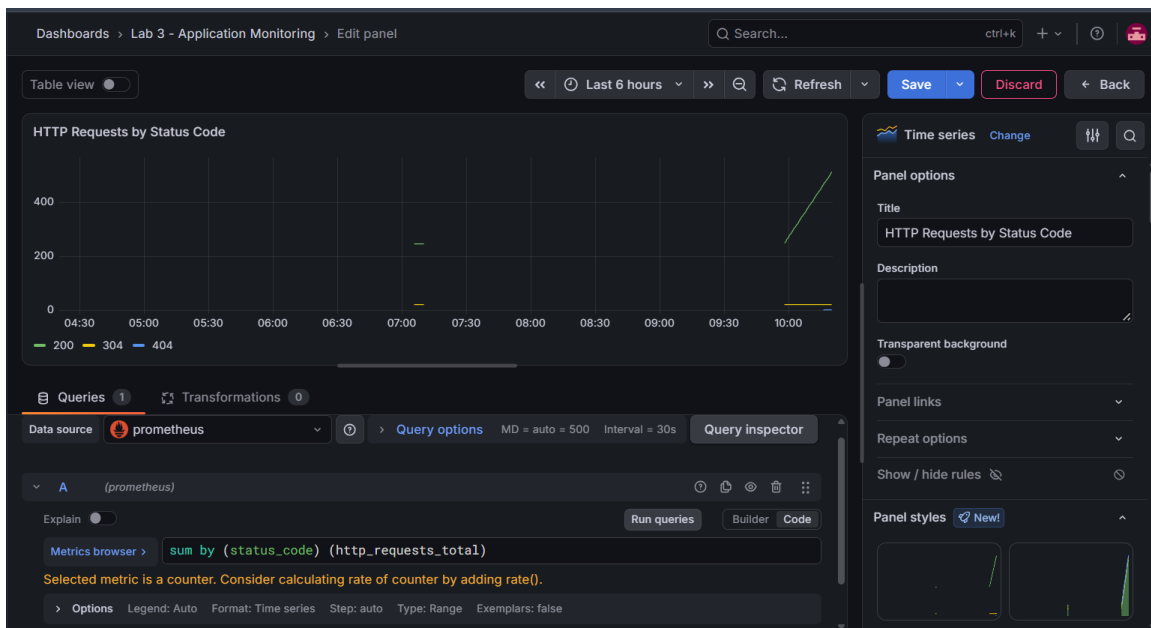
Total HTTP Requests panel



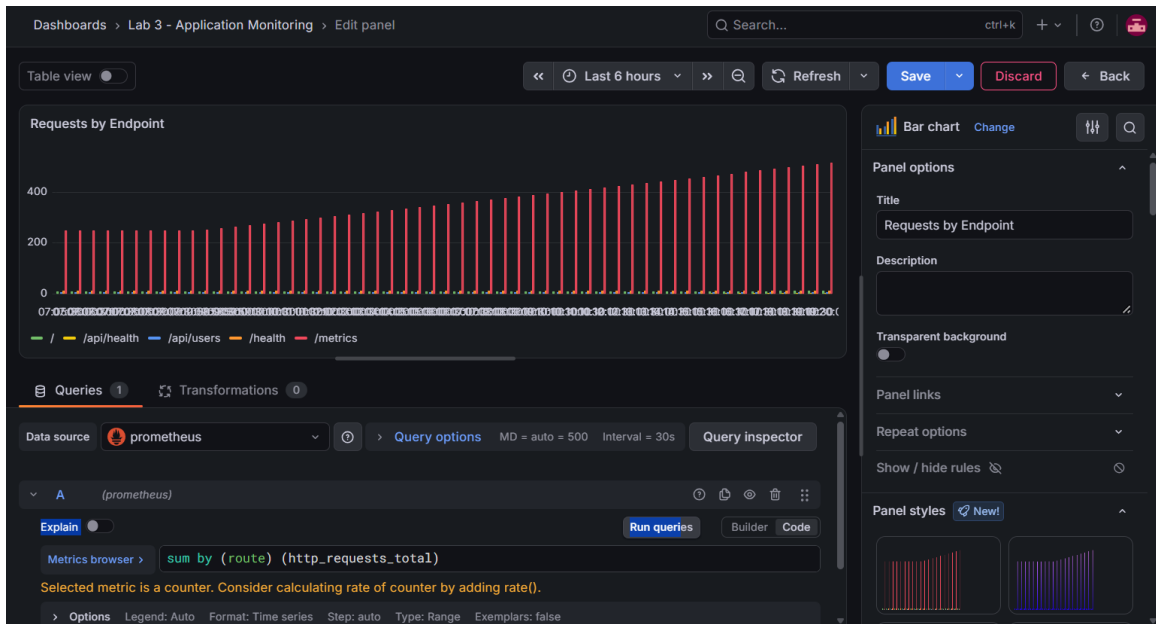
HTTP Request Rate panel



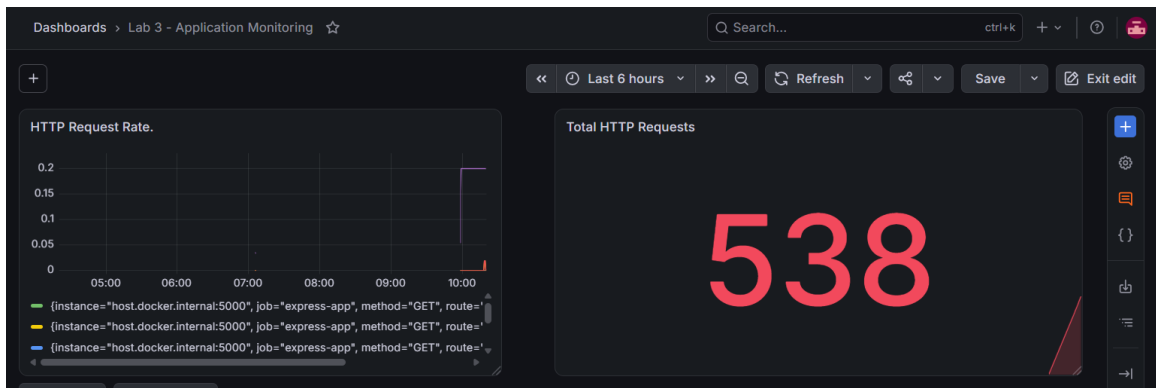
Memory panel

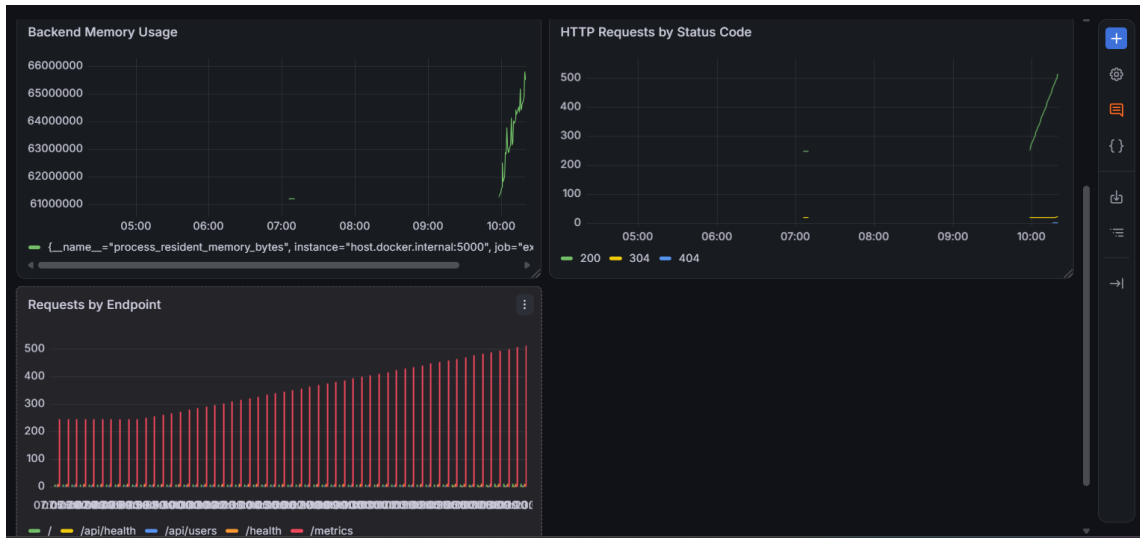


Status Panel



Requests by Endpoint panel





Final Grafana dashboard